Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

Report on laboratory work no. 7

Course: Operating Systems

Topic: Three Small Programs: Processes, Redirection, a Pipe

Done by:
st. gr. FAF-243, Ilie Covali

Checked by:
Reițman Patricia

Chișinău – 2026

Operating Systems · FCIM / FAF · UTM · 2026-2027

Lab 7 · Three Small Programs: Processes, Redirection, a Pipe

Metadata & Environment

- **Course:** Operating Systems, FCIM / FAF, UTM, Academic Year 2026-2027
- **Lab Work:** Lab 7 (Track A: Three Small Programs: Processes, Redirection, a Pipe, Linux)
- **Author:** Ilie Covali (st. gr. FAF-243)
- **Verified by:** Reițman Patricia
- **Host OS:** Linux (CachyOS x86_64, Kernel 7.2.9-2-cachyos)
- **Compiler:** GCC 16.2.1 20260810 (`gcc -Wall -Wextra -O2`)
- **Shell:** GNU bash, version 5.3.20(1)-release (x86_64-pc-linux-gnu)
- **Workspace:** `/home/ilie/Documents/SO/os-lab7` (`~/os-lab7`)

Purpose of the Work

1. Examine the fundamental Unix process lifecycle primitives: process replication (`fork`), program image replacement (`exec`), and state synchronization (`waitpid`).
2. Induce and investigate process termination states in the Linux kernel, specifically the creation, detection, and lifecycle of a **zombie process** (`Z` / `<defunct>`).
3. Analyze file descriptor table mechanics and verify the POSIX lowest-numbered free descriptor allocation rule used to implement shell output redirection (`>`).
4. Implement inter-process communication (IPC) through a unidirectional anonymous kernel pipe (`pipe`), managing file descriptor duplication (`dup`) and reference counting.
5. Demonstrate the critical role of descriptor closure in pipe synchronization, observing how unclosed write descriptors stall reader processes awaiting EOF (`read(2)` blocking).

Part 1. `fork` + `exec` + `wait`

1. Architectural Concept

When an interactive Unix shell executes a command such as `ls -l`, it does not execute the utility within its own process context. Instead, it performs a three-stage sequence: 1. **`fork()`**: Clones the caller process, creating a new child process with an identical address space, copied file descriptor table, and distinct Process ID (`PID`). 2. **`exec()`** (e.g., `execlp()`): Replaces the child's virtual address space (text, initialized data, BSS, heap, and stack) with a newly loaded executable image from disk. 3. **`wait()` / `waitpid()`**: Blocks the parent process until the child changes its state (terminates or stops), reading its termination status code to prevent zombie leakage.

2. Implementation: `run.c`

The program implements this canonical triad:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* the three steps of the shell for the command: ls -l */
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork(); /* step 1: copy myself */
    if (pid < 0) {
        perror("fork");
        return 1;
```

```c
    }

    if (pid == 0) { /* only the child runs this */
        printf("child: my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL); /* step 2: become ls */
        perror("exec");                         /* runs only if exec failed */
        exit(127);
    }

    int status;
    waitpid(pid, &status, 0); /* step 3: wait for the child */
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

3. Compilation & Execution

Compiling with strict diagnostic flags and executing the baseline program:

```
❯ gcc -Wall -Wextra -O2 -o run run.c
❯ ./run
parent: my PID is 659785
child: my PID is 659786, my parent is 659785
total 168
lrwxrwxrwx 1 ilie ilie    11 Oct 19 13:56  build.sh -> ../build.sh
lrwxrwxrwx 1 ilie ilie    16 Oct 19 13:56  labreport.cls -> ../labreport.cls
-rw-r--r-- 1 ilie ilie   378 Oct 19 13:57  Makefile
-rw-r--r-- 1 ilie ilie 35137 Oct 19 13:51 'OS 2026 - Lab 7 - Three Small Programs (Linux).docx'
-rwxr-xr-x 1 ilie ilie 16408 Oct 19 13:57  pipeline
-rw-r--r-- 1 ilie ilie   893 Oct 19 13:57  pipeline.c
-rwxr-xr-x 1 ilie ilie 16448 Oct 19 13:57  redirect
-rw-r--r-- 1 ilie ilie   768 Oct 19 13:57  redirect.c
-rwxr-xr-x 1 ilie ilie 16512 Oct 19 13:57  run
-rw-r--r-- 1 ilie ilie   851 Oct 19 13:57  run.c
-rwxr-xr-x 1 ilie ilie 16560 Oct 19 13:57  zombie
-rw-r--r-- 1 ilie ilie   911 Oct 19 13:57  zombie.c
parent: child 659786 ended with status 0
```

4. Alternative Program Executions

To verify the behavior of `execlp()` across different programs and failure conditions:

Sub-experiment A: Running `date` Substituting `execlp("date", "date", (char *)NULL);` in the child branch:

```
❯ ./run_date
parent: my PID is 659838
child: my PID is 659839, my parent is 659838
Mon Oct 19 01:57:52 PM EEST 2026
parent: child 659839 ended with status 0
```

The child successfully metamorphoses into `/usr/bin/date`, prints the formatted timestamp, and terminates cleanly with exit code 0.

Sub-experiment B: Running `nosuchprog` (Exec Failure) Substituting `execlp("nosuchprog", "nosuchprog", (char *)NULL);` in the child branch:

```
❯ ./run_nosuch
parent: my PID is 659846
child: my PID is 659847, my parent is 659846
exec: No such file or directory
parent: child 659847 ended with status 127
```

Because `nosuchprog` does not exist in any directory listed in `$PATH`, `execlp` fails and returns `-1`. The child process falls through to `perror("exec")`, prints the corresponding `ENOENT` diagnostic string (`No such file or directory`) to `stderr`, and calls `exit(127)`.

Observe (Part 1 Answers)

1. **The line `perror("exec")` is in the child, right after `execlp`. Why is it printed only with `nosuchprog`?** The `exec` family of system calls (`execlp`, `execvp`, `execve`) does not return on success. When `execlp()` succeeds, the kernel overwrites the entire virtual address space of the calling child process with the code, stack, and data segments of the target binary (`ls` or `date`). The original program code following the call ceases to exist in memory. If and only if the `exec` call encounters an error (e.g., binary not found, invalid executable format, missing permissions), `execlp()` returns `-1` to the caller, allowing subsequent instructions—`perror("exec")` and `exit(127)`—to execute.
2. **Which number does the child print as "my parent", and whose PID is it?** The child prints `my parent is 659785`, retrieved via the `getppid()` system call. This number corresponds exactly to the Process ID (`PID`) of the parent process that invoked `fork()` (verified by the parent's `getpid()` output `parent: my PID is 659785`).
3. **What status does the parent see with `ls`, and what status with `nosuchprog`?**
   - With `ls`, the parent observes status **`0`**, indicating that `ls` executed and exited normally without runtime errors (`EXIT_SUCCESS`).
   - With `nosuchprog`, the parent observes status **`127`**. This status was explicitly emitted by the child via `exit(127)` following the `perror("exec")` failure branch, following standard POSIX shell convention where exit code 127 signifies "command not found".

Part 2. A Zombie, on Purpose

1. Theoretical Mechanism of Zombie Processes

In Unix-like systems, process termination is a two-phase protocol: 1. When a process terminates (by calling `exit()` or being terminated by a signal), the kernel deallocates its virtual memory pages, file descriptor structures, and execution context. 2. However, the process cannot be completely removed from the kernel's process table (`task_struct` in Linux). The kernel retains a minimal bookkeeping stub holding the process's PID, termination status (`wstatus`), and resource usage statistics (`rusage`). 3. While in this state, the process is classified as a **zombie** (`TASK_ZOMBIE` / state `Z`). 4. Only when the parent process reads the termination status via `wait()` or `waitpid()` is the zombie entry deallocated and the PID returned to the system PID pool (known as *reaping*).

2. Implementation: `zombie.c`

We introduce an intentional 20-second window where the parent sleeps before invoking `waitpid()`:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* creates a zombie child process by delaying parent waitpid */
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork(); /* step 1: copy myself */
    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) { /* only the child runs this */
```

```
        printf("child: my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL); /* step 2: become ls */
        perror("exec");
        exit(127);
    }

    int status;
    sleep(20);                /* child finishes immediately while parent sleeps -> zombie */
    waitpid(pid, &status, 0); /* step 3: wait for the child and reap zombie */
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

3. Compilation & Execution Trace

We start `./zombie` in the background with stdout redirected to `/dev/null` to keep the terminal clean, inspect the process table after 2 seconds, and inspect it again after the 20-second sleep expires:

```
❯ gcc -Wall -Wextra -O2 -o zombie zombie.c
❯ ./zombie > /dev/null &
❯ sleep 2; ps -o pid,ppid,stat,cmd -g $$
=== PS DURING SLEEP (ZOMBIE ACTIVE) ===
    PID    PPID STAT CMD
 659860  658627 S<s+ bash -c ./zombie > /dev/null & ...
 659861  659860 S<+  ./zombie
 659863  659861 Z<+  [ls] <defunct>
 659864  659860 R<+  ps -o pid,ppid,stat,cmd -g 659860
```

After waiting for the 20-second sleep interval to finish:

```
❯ sleep 19; ps -o pid,ppid,stat,cmd -g $$
=== PS AFTER WAITPID (REAPED) ===
    PID    PPID STAT CMD
 659860  658627 R<s+ ps -o pid,ppid,stat,cmd -g 659860
```

Observe (Part 2 Answers)

1. **Which letter does `ls` have in the `STAT` column, and what does it mean?** `ls` has the status letter **`Z`** (specifically reported as `Z<+`, denoting a high-priority foreground process in state Z). In Linux `ps(1)`, `Z` denotes a **Zombie / Defunct** process. It indicates that the process has completed execution via `exit_group()`, but its entry remains in the kernel process table waiting for its parent process (`PID 659861`) to reap it.
2. **After 20 seconds, is the `[ls] <defunct>` line still there? Which line of `zombie.c` made it disappear?** No, after 20 seconds the `[ls] <defunct>` line is completely absent from the process table. The line of code that caused it to disappear is:

   ```
   waitpid(pid, &status, 0);
   ```

   Calling `waitpid()` retrieves the child's termination status from the kernel, satisfying the parent-child synchronization contract. Once the parent acknowledges the child's termination, the kernel immediately frees the child's `task_struct`, releasing its PID back to the OS.

Part 3. Redirection: `ls -l > out.txt`

1. File Descriptor Allocation Semantics

In Unix, every process possesses a private File Descriptor (`FD`) table. By default: - `0`: Standard Input (`stdin`) - `1`: Standard Output (`stdout`) - `2`: Standard Error (`stderr`)

The POSIX specification guarantees that any system call allocating a new file descriptor (such as `open`, `creat`, or `dup`) **must return the lowest-numbered file descriptor currently unused by the process**.

To implement shell redirection (`ls -l > out.txt`), a process: 1. Closes file descriptor `1` (`close(1)`). 2. Calls `open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644)`. 3. Because descriptor `0` is in use, descriptor `1` is now free, and descriptor `2` is in use, the kernel deterministically assigns index `1` to `out.txt`. 4. When `execlp("ls", ...)` runs, `ls` executes without any modification. It writes all standard output to descriptor 1, which the kernel routes to `out.txt`.

2. Implementation: `redirect.c`

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command: ls -l > out.txt */
int main(void)
{
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        close(1); /* descriptor 1 is now free */
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) {
            perror("open");
            exit(1);
        }
        fprintf(stderr, "open gave descriptor %d\n", fd);
        execlp("ls", "ls", "-l", (char *)NULL); /* ls writes to 1, as always */
        perror("exec");
        exit(127);
    }

    waitpid(pid, NULL, 0);
    printf("done: the listing is in out.txt\n");
    return 0;
}
```

3. Compilation & Verification

Compiling and executing `redirect.c`, followed by inspecting the generated `out.txt`:

```
❯ gcc -Wall -Wextra -O2 -o redirect redirect.c
❯ ./redirect
open gave descriptor 1
done: the listing is in out.txt

❯ head -n 5 out.txt
total 168
lrwxrwxrwx 1 ilie ilie    11 Oct 19 13:56 build.sh -> ../build.sh
lrwxrwxrwx 1 ilie ilie    16 Oct 19 13:56 labreport.cls -> ../labreport.cls
-rw-r--r-- 1 ilie ilie   378 Oct 19 13:57 Makefile
-rw-r--r-- 1 ilie ilie 35137 Oct 19 13:51 OS 2026 - Lab 7 - Three Small Programs (Linux).docx
```

The diagnostic output to `stderr` confirms that `open()` returned descriptor `1`. The terminal displayed none of the directory listing lines; all 168 KB of directory content was captured inside `out.txt`.

Observe (Part 3 Answers)

1. **Which number did `open` give, and why exactly that one?** `open()` gave descriptor **`1`**. According to the POSIX standard (`open(2)` specification), the kernel must assign the lowest-numbered file descriptor that is currently unallocated in the process's file descriptor table. Because descriptor

`0` was open (`stdin`), descriptor `1` was explicitly released via `close(1)`, and descriptor `2` was open (`stderr`), slot 1 was the lowest available integer index.

2. **`ls` was not changed at all. Why does its listing not appear on the screen?** Standard user-space programs like `ls` do not contain hardcoded knowledge of display terminals. They simply invoke the `write()` system call specifying file descriptor `1` (`STDOUT_FILENO`). Because file descriptors remain open across `exec` family calls (unless marked with `FD_CLOEXEC`), descriptor `1` pointed to the newly opened file `out.txt` on disk. When `ls` wrote its output buffer to descriptor 1, the kernel file subsystem directed those byte streams into `out.txt` instead of the terminal virtual console device.

3. **To redirect the errors instead (like `2> err.txt`), which descriptor would you close before `open`?** To redirect standard error, you would close file descriptor **`2`** (`STDERR_FILENO`) before calling `open()`.

Part 4. A Pipe: `ls /etc | wc -l`

1. Pipeline IPC Architecture

A Unix pipeline connects the standard output of an upstream process to the standard input of a downstream process through a circular in-memory buffer maintained by the operating system: - `pipe(int p[2])` allocates a pipe object: - `p[0]`: The **read end** of the pipe. - `p[1]`: The **write end** of the pipe. - Using `dup()` (or `dup2()`), each child maps the appropriate pipe endpoint to standard stream descriptors (0 or 1) and closes all unused endpoints. - Proper reference count management is essential: a reader process receives an **End-Of-File (`EOF`)** marker (return value 0 from `read()`) if and only if **all** file descriptors referencing the write end (`p[1]`) across the entire system are closed.

2. Implementation: `pipeline.c`

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command: ls /etc | wc -l */
int main(void)
{
    int p[2];
    if (pipe(p) < 0) {
        perror("pipe");
        return 1;
    }

    if (fork() == 0) { /* ls: its 1 becomes p[1] */
        close(1);
        dup(p[1]);     /* smallest free number: 1 */
        close(p[0]);
        close(p[1]);
        execlp("ls", "ls", "/etc", (char *)NULL);
        perror("exec ls");
        exit(127);
    }

    if (fork() == 0) { /* wc: its 0 becomes p[0] */
        close(0);
        dup(p[0]);     /* smallest free number: 0 */
        close(p[0]);
        close(p[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
        exit(127);
    }
```

```
    close(p[0]); /* the parent uses neither end */
    close(p[1]);
    wait(NULL);
    wait(NULL);
    return 0;
}
```

3. Compilation & Output Equivalence Check

We compile and run `./pipeline` and compare its output directly against the native shell command `ls /etc | wc -l`:

```
❯ gcc -Wall -Wextra -O2 -o pipeline pipeline.c
❯ ./pipeline
259
❯ ls /etc | wc -l
259
```

Both commands return `259`, confirming exact data transport and synchronization.

4. Intentional Breakage Experiment: The Hanging Pipe

We test the synchronization model by intentionally commenting out the parent's write-end closure:

```
    close(p[0]); /* the parent uses neither end */
    /* close(p[1]); */
    wait(NULL);
    wait(NULL);
```

When compiled and executed:

```
❯ gcc -Wall -Wextra -o pipeline_broken pipeline_broken.c
❯ ./pipeline_broken
# The process hangs indefinitely until interrupted by SIGINT (Ctrl-C)
^C
```

Observe (Part 4 Answers)

1. **Did `./pipeline` and `ls /etc | wc -l` give the same number?** Yes. Both `./pipeline` and `ls /etc | wc -l` produced the exact value **`259`**.
2. **After the change, which of the two children never ends, `ls` or `wc`? `wc`** never ends (`ls` finishes its execution and exits immediately after listing `/etc`).
3. **Why, in one sentence: what is `wc` waiting for?**
4. **IPC Pipelines & Reference Counting**: We constructed a two-stage pipeline using an anonymous pipe and descriptor duplication. Crucially, we demonstrated that pipe data streams rely on strict write-end reference counts: omitting a single `close()` call on a write descriptor prevents reader processes from receiving EOF, resulting in indefinite execution deadlocks.